

BarrierShare Results

一、最终实验优先级

优先级	表号	实验名称	是否主文	作用	为什么是这个优先级
P0 必跑	表 1	LLM 主结果表	是	主 empirical 结果	这是整篇论文最核心的真实环境/LLM 结果，直接回答 “PS 是否优于 baseline”
P0 必跑	表 4	仿真主实验	是	理论主对齐	对齐 regret 主线，验证 all-share / partial-share / all-unshare 的整体趋势
P0 必跑	表 5	long safe suffix 实验	是	结构优势主证据	这是你当前 theorem 最关键的结构点，直接验证 (R<L) 时的优势来源
P0 必跑	表 6	不同深度实验	是	深度效应主证据	你已经明确想做，而且它能直接展示多阶段系统变深时 PS 和 baseline 的差异
P1 建议跑	表 2	LLM 机制消融结果表	是/可压缩	机制解释	回答 algorithm_direct / theta_guided_agent / agent_only 三种机制下，PS 信息是否还能转化为收益
P1 建议跑	表 3	candidate 数扩展表	是	扩展性	固定 4/5-share、5/15/25、五种算法全对比，很适合展示 branching 变大后的稳定性
P2 可选	表 7	specialist mass 实验	附录优先	机制补充	能解释什么时候 private/unshared specialist 分支更重要，但不是主 theorem 必需
P2 可选	表 8	noise 实验	附录优先	鲁棒性补充	有价值，但优先级低于 safe suffix 和 depth

A. LLM实验设置表

LLM 实验指标

列名	含义	统计方式	是否建议放主表
Avg Total Cost ↓	每轮最终总代价的平均值，主排序指标	对所有 round / episode 的 total cost 取均值	是
Success / EM ↑	任务成功率或精确匹配率	成功轮次 / 总轮次	是
Terminal Penalty ↓	最终失败、偏航、未达到目标终态带来的惩罚	对每轮 terminal penalty 取均值	是
Path Cost ↓	由路径选择本身带来的代价部分	对每轮 path cost component 取均值	是
LLM Calls ↓	每轮平均调用 LLM 的次数	对每轮调用次数取均值	是
SharedFrac	最终走到 shared leaf 的轮次占比	# shared leaf rounds / T	否，建议机制表
UnsharedFrac	最终走到 unshared leaf 的轮次占比	# unshared leaf rounds / T	否，建议机制表
SharedUpdFrac	触发 shared update 的轮次占比	shared update rounds / T	否，建议机制表
SharedUpdCnt	一共触发了多少次 shared update	原始计数	否，appendix 用
CumSharedUpd	截止某时刻累计发生的 shared update 次数	时间序列或最终累计值	否，更适合画曲线

表 1：LLM 主结果表

Setting	Method	Share Ratio	Avg Total Cost ↓	EM / Success ↑	Path Cost ↓	Terminal Penalty ↓
Fixed-stage	PS	all-share				
Fixed-stage	PS	4/5-share				
Fixed-stage	PS	2/5-share				
Fixed-stage	PS	all-unshare				
Fixed-stage	EXP3	ref tree				
Fixed-stage	(\epsilon)-EXP3	ref tree				
Fixed-stage	Random	ref tree				
Fixed-stage	Naive	ref tree				

说明

LLM 主结果表不把 regret 放进主表，主排序用 end-to-end cost / success。

表 2：LLM 机制消融结果表

Mechanism	Method	Tree	Candidate / Stage	Avg Total Cost ↓	Success / EM ↑	Terminal Penalty ↓
theta_guided_agent	PS	4/5-share	5			
agent_only	PS	4/5-share	5			

表 3：candidate 数扩展表

固定 4/5-share，candidate 数量 5 / 15 / 25，五种算法全部比较。

Method	Tree	Candidate / Stage	Avg Total Cost ↓	Success / EM ↑	Terminal Penalty ↓	Path Cost ↓
PS	4/5-share	5				
PS	4/5-share	15				
PS	4/5-share	25				
EXP3	4/5-share	5				
EXP3	4/5-share	15				
EXP3	4/5-share	25				
(\epsilon)-EXP3	4/5-share	5				
(\epsilon)-EXP3	4/5-share	15				
(\epsilon)-EXP3	4/5-share	25				
Random	4/5-share	5				
Random	4/5-share	15				
Random	4/5-share	25				
Naive	4/5-share	5				
Naive	4/5-share	15				
Naive	4/5-share	25				

B. Simulation实验设置表

Sim实验指标

列名	含义	统计方式	是否建议放主表
Reg_T ↓	到 horizon (T) 为止的累计 regret	直接累计	是
Reg_T / T ↓	平均每轮 regret	Reg_T / T	是
AvgCost ↓	每轮实际 cost 的平均值	对所有轮次 cost 取均值	是
BestPath@T ↑	最终识别/收敛到最优固定路径的比例	# runs ending on best path / # runs 或末段窗口统计	是
SharedFrac	最终走到 shared leaf 的轮次占比	# shared leaf rounds / T	是
SharedUpdFrac	触发 shared update 的轮次占比	shared update rounds / T	是
UnsharedFrac	最终走到 unshared leaf 的轮次占比	# unshared leaf rounds / T	否，可选
SharedUpdCnt	一共触发了多少次 shared update	原始计数	否，appendix 用
CumSharedUpd	截止某时刻累计 shared updates	时间序列或最终累计值	否，更适合画曲线

表 4：仿真主实验

5层，每层每节点各连5个孩子节点

Tree	Candidate / Stage	Method	T	Reg_T ↓	Reg_T / T ↓	AvgCost ↓
all-share	5	PS	10^7			
4/5-share	5	PS	10^7			
2/5-share	5	PS	10^7			
all-unshare	5	PS	10^7			
4/5-share	5	EXP3	10^7			
4/5-share	5	ϵ -EXP3	10^7			
4/5-share	5	Random	10^7			
4/5-share	5	Naive	10^7			

表 5：long safe suffix实验

这个表对应“当 $(R < L)$ 时指数从 $(L/(L+1))$ 降到 $(R/(R+1))$ ”的情况。
 严格按你当前 theorem 和 barrier/root regret 定义，其实是 **long safe suffix**，因为是“前面 risky，后面接一个完整 safe 子树”。
 建议固定：

- 总深度 (L) 固定，比如 8
- 每层分支数 ($d_i = 4$)
- risky 层每个节点只有 1 个真正危险孩子，即 ($D^U(i)=1$)
- 通过改变 (R) 改变 safe suffix 长度

Case	Total Depth (L)	Risky Depth (R)	Branching (d_i)	($D^U(i)$)	Safe Suffix Length (L-R)	Method	Reg_T ↓	Re
SS-0	8	8	4	1	0	PS		
SS-1	8	6	4	1	2	PS		
SS-2	8	4	4	1	4	PS		
SS-3	8	2	4	1	6	PS		
SS-0	8	8	4	1	0	ϵ -EXP3		
SS-1	8	6	4	1	2	ϵ -EXP3		
SS-2	8	4	4	1	4	ϵ -EXP3		
SS-3	8	2	4	1	6	ϵ -EXP3		

表 6：不同深度实验

- **PS**: 深度 sweep 时，跑 all-share / 4/5-share / 2/5-share / all-unshare
- **ϵ -EXP3 / EXP3 / Naive / Random**: 不同深度下只跑一次
- 固定每层候选数 = 5
- 深度 sweep: 5 / 15 / 25

Tree	Depth (L)	Candidate / Stage	Method	Reg_T ↓	Reg_T / T ↓	AvgCost ↓
all-share	5	5	PS			
all-share	15	5	PS			
all-share	25	5	PS			
4/5-share	5	5	PS			
4/5-share	15	5	PS			
4/5-share	25	5	PS			
2/5-share	5	5	PS			
2/5-share	15	5	PS			
2/5-share	25	5	PS			
all-unshare	5	5	PS			
all-unshare	15	5	PS			
all-unshare	25	5	PS			
4/5-share	5	5	ε-EXP3			
4/5-share	15	5	ε-EXP3			
4/5-share	25	5	ε-EXP3			
4/5-share	5	5	EXP3			
4/5-share	15	5	EXP3			
4/5-share	25	5	EXP3			
4/5-share	5	5	Naive			
4/5-share	15	5	Naive			
4/5-share	25	5	Naive			
4/5-share	5	5	Random			
4/5-share	15	5	Random			
4/5-share	25	5	Random			

下面的表7表8都是大模型给我的，可以一起看看需不需要

表 7：specialist mass 实验

这个实验回答的是：

当任务越来越多地落到 “private / specialist / unshared 分支” 上时，PS 会不会更占优？

我建议把 specialist mass 定义成：

$$p_{\text{spec}} = \text{Pr}(\text{本轮任务最终踩到 unshare})$$

然后 sweep：

$p_{\text{spec}} \in 0, 0.25, 0.5, 0.75, 1.$

Case	(p_{spec})	Tree Template	Candidate / Stage	Method	Reg_T ↓	Reg_T / T ↓	AvgCost
SM-0	0	fixed	5	PS			
SM-1	0.25	fixed	5	PS			
SM-2	0.5	fixed	5	PS			
SM-3	0.75	fixed	5	PS			
SM-4	1	fixed	5	PS			
SM-0	0	fixed	5	ϵ -EXP3			
SM-1	0.25	fixed	5	ϵ -EXP3			
SM-2	0.5	fixed	5	ϵ -EXP3			
SM-3	0.75	fixed	5	ϵ -EXP3			
SM-4	1	fixed	5	ϵ -EXP3			

表 8: noise 实验

这个实验回答的是：

当 terminal cost 带噪声 / 执行不稳定时，PS 是否仍然稳？

最干净的设计是 bounded additive noise：

$$c_t(\ell) = \text{clip}(c_t^*(\ell) + \xi_t, 0, 1), \quad \xi_t \sim \text{Unif}[-\sigma, \sigma].$$

建议 sweep：

$$\sigma \in 0, 0.05, 0.1, 0.2.$$

Case	Noise Level (σ)	Tree	Candidate / Stage	Method	Reg_T ↓	Reg_T / T ↓	AvgCost
N-0	0	4/5-share	5	PS			
N-1	0.05	4/5-share	5	PS			
N-2	0.1	4/5-share	5	PS			
N-3	0.2	4/5-share	5	PS			
N-0	0	4/5-share	5	ϵ -EXP3			
N-1	0.05	4/5-share	5	ϵ -EXP3			
N-2	0.1	4/5-share	5	ϵ -EXP3			
N-3	0.2	4/5-share	5	ϵ -EXP3			

如果你更想要 agent 味道，可以把 (σ) 解释成 execution instability level。